

En este capítulo aprenderemos los elementos básicos del lenguaje de programación Python y por tanto, una vez lo completemos ya podremos escribir nuestros primeros programas. Es muy importante, pues, que tengas ya instalado el Python (versión 3) y un buen editor de textos.

De este capítulo dispones de los siguientes archivos:

- El propio capítulo en PDF: [02-Elementos_basicos-spa.pdf](#)
- El notebook de Jupyter: [cap2-spa.ipynb](#) que necesita el archivo [archivo.txt](#)
- Los programas: [programas2-spa.zip](#)
- Enlace al notebook en Google Colab: <https://colab.research.google.com/github/aoliverg/python/blob/master/notebooks/cap2-spa.ipynb>

2.1. Tipos básicos, operaciones y conversión entre tipos

2.1.a. Tipos básicos

Un concepto muy importante en informática es el de *variable*, que es un nombre simbólico asociado a un valor que puede cambiar. Por ejemplo, si en Python escribimos:

```
a=3
```

estamos asignando el valor 3 a la variable a. Los datos que pueden almacenar las variables pueden ser de diferente tipo: números, cadenas, valores lógicos, etc. A pesar de que no es necesario declarar qué tipo de dato almacena una variable (recuerda que en el apartado 1.1. hablábamos del *tipado dinámico* como una de las características de Python), una vez asignamos un valor a una determinada variable, esta automáticamente tendrá un tipo determinado que se mantendrá durante la ejecución del programa, si no es que le asignamos más adelante un valor de otro tipo a esta misma variable.

Los tipos principales en Python son los siguientes:

- **Valores booleanos** que pueden tomar los valores Cierto (True) o Falso (False).
- **Números** que pueden ser enteros (integers) (1 y 2), de coma flotante (floats) (1.1 y 1.2), o complejos (complex) (3j+2)
- **Cadenas (strings)** son secuencias de caracteres Unicode ("me llamo Antoni" o "Меня зовут Антон")
- **Listas (lists)**, son secuencias ordenadas de valores (["lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"].
- **Colecciones (sets)** son un conjunto de valores sin orden. El ejemplo anterior de la lista nos serviría, pero teniendo en cuenta que no tendrían un orden determinado
- **Diccionarios (dictionaries)**: son un conjunto sin orden de pares clave-valor. Por ejemplo edad["Paula"]=27; edad["Joan"]=15; edad[Pau]=32, que también se podría representar como edad={'Joan': 15, 'Paula': 27, 'Pau': 32}. A partir de la versión 3.7 de Python los diccionarios mantienen el orden de inserción.

La función type nos devuelve el tipo de un objeto. Ahora abre un intérprete interactivo y escribe las siguientes instrucciones:

```
>>> a=True
>>> print(a)
True
>>> type(a)
<class 'bool'>
```

Fíjate que type nos devuelve el tipo de la variable. Cómo que en Python3 todo es una clase nos dice class

```
>>> a=3
>>> type(a)
```

```
<class 'int'>
```

Fijaos que en la misma sesión del intérprete interactivo has vuelto a usar la variable `a` para otro tipo y que se ha cambiado el tipo de la variable de manera dinámica.

```
>>> a=1.1
```

```
>>> type(a)
```

```
<class 'float'>
```

```
>>> a=3j+2
```

```
>>> type(a)
```

```
<class 'complex'>
```

```
>>> a="me llamo Antoni"
```

```
>>> type(a)
```

```
<class 'str'>
```

```
>>> a="Меня зовут Антон"
```

```
>>> type(a)
```

```
<class 'str'>
```

(si no puedes escribir en ruso con el teclado intenta copiar la cadena en el intérprete)

```
>>> a=["lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"]
```

```
>>> type(a)
```

```
<class 'list'>
```

Podemos hacer cosas del estilo

```
>>> print(a[0])
```

```
lunes
```

La primera posición es la 0

```
>>> print(a[1])
```

```
martes
```

Y la segunda la 1.

En este caso la última será la 6, puesto que tenemos 7 elementos.

```
>>> print(a[6])
```

```
domingo
```

También podemos acceder a la última posición con el índice -1:

```
>>> print(a[-1])
domingo
```

Si intentamos acceder a una posición no existente, se produce un error:

```
>>> print(a[7])
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

Ahora convertiremos la lista a en un set.

```
>>> a=set(a)
>>> type(a)
<class 'set'>
>>> print(a)
{'sábado', 'jueves', 'lunes', 'martes', 'miércoles', 'viernes', 'domingo'}
```

Fíjate que al pasar a a set se ha perdido el orden. Además si intentamos acceder a una posición concreta se produce un error:

```
>>> print(a[0])
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: 'set' object does not support indexing
```

Para empezar a trabajar con un diccionario la mejor manera es primero declararlo:

```
>>> a={}
```

Y podemos ver el tipo de la variable haciendo:

```
>>> type(a)
<class 'dict'>
```

Ahora ya podemos hacer:

```
>>> edad={}
>>> edad["Paula"]=27
>>> edad["Joan"]=15
>>> edad["Pau"]=32
```

Y si queremos escribir todo el diccionario, escribimos:

```
>>> print(edad)
```

```
{'Joan': 15, 'Paula': 27, 'Pau': 32}
```

Pero si queremos acceder a una edad determinada escribimos:

```
>>> print(edad["Paula"])
```

```
27
```

Si intentamos recuperar el valor para una clave no existente se produce un error:

```
>>> print(edat["Antoni"])
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
KeyError: 'Antoni'
```

Ahora ya conocemos los principales tipos en Python. Cada uno de estos tipos tendrán asociadas una serie de operaciones posibles, pero las iremos explicando a medida que las vamos necesitando en el resto de secciones.

2.1.b. Operaciones básicas

Con los tipos numéricos podemos hacer las operaciones habituales, sumas, restas, etc.

```
>>> a=3
```

```
>>> b=2
```

```
>>> c=a+b
```

```
>>> print(c)
```

```
5
```

```
>>> d=a/b
```

```
>>> print(d)
```

```
1.5
```

Podemos incrementar o decrementar el valor de una variable de las siguientes maneras:

```
>>> a=1
```

```
>>> a=a+1
```

```
>>> print(a)
```

```
2
```

```
>>> a=1
```

```
>>> a+=1
```

```
>>> print(a)
```

```
2
```

Las cadenas las podemos concatenar con el operador "+".

```
>>> a="hola"
>>> b="buenos días"
>>> d=a+" "+b
>>> print(d)
hola buenos días
```

Pero cuidado, no podemos usar el operador "+" mezclando valores numéricos y cadenas.

```
>>> a="tengo"
>>> b=3
>>> c="manzanas"
>>> d=a+" "+b+" "+c
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: Can't convert 'int' object to str implicitly

Para poder hacerlo tendremos que convertir el valor numérico en cadena (este tipo de conversiones las veremos en el siguiente subapartado, pero ahora avanzamos que para convertir de integer a cadena podemos utilizar `str()`):

```
>>> d=a+" "+str(b)+" "+c
>>> print(d)
tengo 3 manzanas
```

2.1.c. Conversión entre tipos

Ya hemos visto un ejemplo en el subapartado anterior de como convertir un valor numérico a cadena usando `str()`. Vemos ahora las conversiones más habituales:

De cadena a entero usando `int()` o a coma flotante usando `float()`. Recordad que el operador "+" aplicado a cadenas las concatena:

```
>>> a="3"
>>> b="2.1"
>>> print(a+b)
32.1
```

Si lo que queremos es obtener la suma de 3 y 2.1 tendremos que convertirlos en los valores numéricos correspondientes:

```
>>> a=int(a)
>>> b=float(b)
>>> print(a+b)
5.1
```

También podríamos haber convertido directamente a float y obtendríamos el mismo resultado.

Recordad el ejemplo que ya hemos visto de conversión de valor numérico a cadena usando `str()`.

En el supuesto de que tengamos una cadena con una serie de valores separados por algún carácter concreto, podemos convertir esta cadena en lista usando `split()`. Fíjate que como parámetro a `split` le damos el separador:

```
>>> a="lunes:martes:miércoles:jueves:viernes:sábado:domingo"
>>> b=a.split(":")
>>> print(b)
['lunes', 'martes', 'miércoles', 'jueves', 'viernes', 'sábado', 'domingo']
```

De manera inversa, podemos convertir una lista en una cadena juntando sus elementos por un separador concreto con `join()`, de la siguiente manera:

```
>>> c=",".join(b)
>>> print(c)
lunes,martes,miércoles,jueves,viernes,sábado,domingo
```

Sería muy largo y aburrido indicar todas las operaciones y funciones disponibles en este subapartado. Con las que hemos presentado será suficiente para empezar a trabajar en aplicaciones reales. A medida que las vamos necesitando, iremos presentando nuevas operaciones y funciones.

2.2. Control de flujo

2.2.a. Introducción

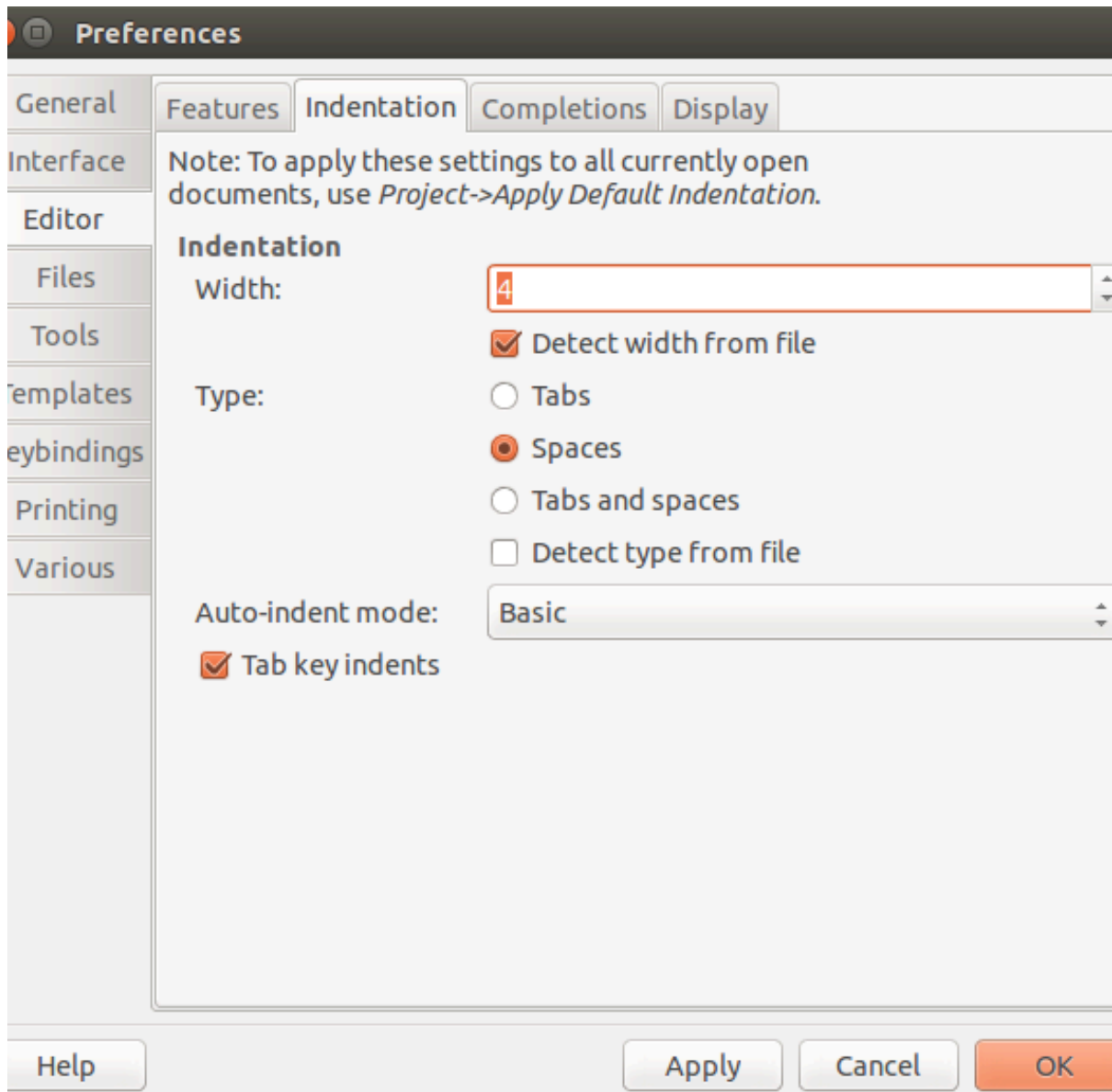
Los programas no son siempre una secuencia de instrucciones que se ejecutan una después de la otra. A menudo hay partes del código que solo se ejecutan si se cumple una condición determinada. También pueden haber instrucciones que se ejecutan repetidamente hasta que se cumple una determinada condición. En este apartado veremos las instrucciones que nos permiten tener el control sobre el flujo de ejecución del programa.

Un aspecto muy importante a tener en cuenta es que los bloques de instrucciones que dependen de alguna instrucción de control de flujo se marcan mediante el sangrado del texto, es decir, dejando un número concreto de espacios delante de las instrucciones que dependen de la instrucción de control de flujo. Es muy importante que el número de espacios sea el mismo en todo el programa (habitualmente 4 espacios). También se puede sustituir este número de espacios por un tabulador. Pongamos un ejemplo:

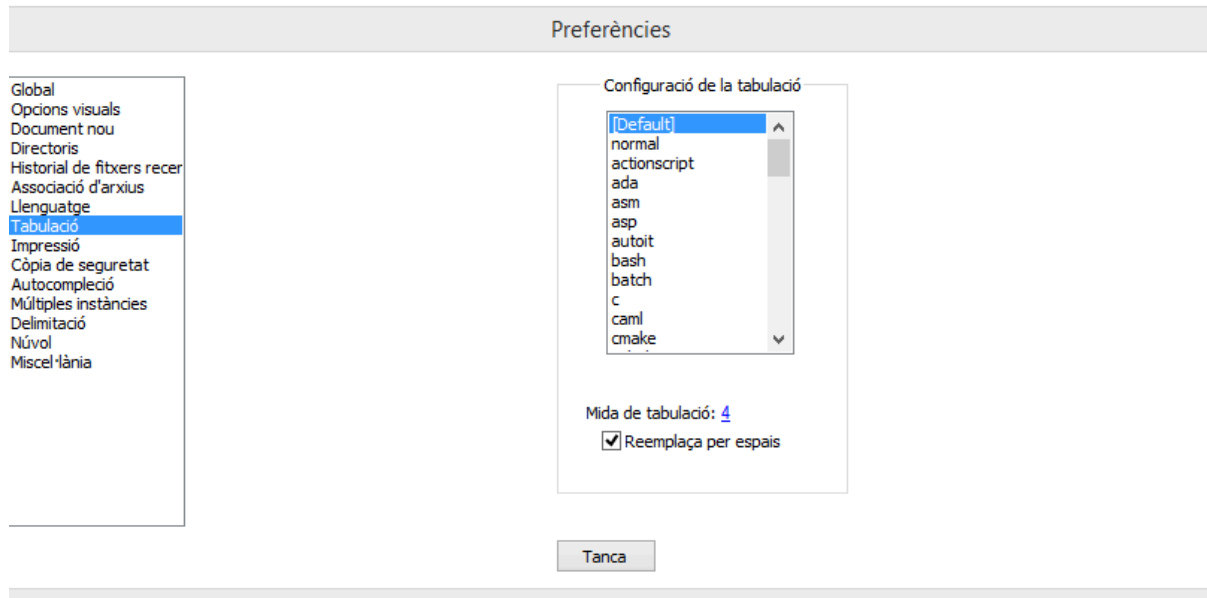
```
a=3
if a<5:
    print("Menor que 5")
else:
    print("Mayor o igual que 5")
```

En este caso, el número de espacios que hay delante de las instrucciones `print` es de 4. Este número de espacios se tendrá que mantener durante todo el fichero de programa. Dado que algunos de los programas los bajaréis de nuestra web y que tendréis que hacer modificaciones, es importante que nos pongamos de acuerdo en cómo tiene que ser el sangrado en nuestros programas. Os propongo que sea de 4 espacios. Para hacerlo, en cualquier editor de textos tendréis que picar 4 espacios seguidos o bien configurar la tecla tabulador porque escriba 4 espacios en lugar de un tabulador. Para hacerlo:

Geany: Id a **Edit > Preferences y Editor > Indentation** y poned las opciones como en la siguiente imagen:



Notepad++: Ir a **Configuración > Preferencias** y seleccionar **Tabulación**. Poner las opciones como en la siguiente imagen:



En otros editores de texto la configuración tiene que ser similar.

2.2.b. Sentencias condicionales: if, if...elif, if...elif...else

La sentencia if nos permite indicar una condición y se puede complementar con elif (sino si) y else (sino). Un ejemplo completo seria:

```
etiqueta="N"
if etiqueta=="N":
    print("Nombre")
elif etiqueta=="V":
    print("Verbo")
elif etiqueta=="A .":
    print("Adjetivo")
elif etiqueta=="R":
    print("Adverbio")
else:
    print("Otros")
```

Si la condición solo afecta a una instrucción no es necesario empezar línea nueva y se puede escribir de manera mucho más compacta como:

```
etiqueta="N"
if etiqueta=="N": print("Nombre")
elif hashtag=="V": print("Verbo")
elif hashtag=="A .":print("Adjetivo")
```



```
elif etiqueta=="R": print("Adverbio")
else: print("Otros")
```

Fijate que en las asignaciones se utiliza un solo signo igual ($A=2$) y en las comparaciones se utilizan dos ($\text{if } A==2:$).

2.2.c. Bucles while

Un bucle es un conjunto de instrucciones que se repiten. Con while podemos hacer que unas instrucciones se repitan mientras se cumpla una determinada condición. Un ejemplo sencillo sería:

```
a=0
while a<=10:
    print(a)
    a+=1
```

La salida de este programa son los números del 0 al 10.

Si la condición se cumple siempre, creamos un bucle infinito. Por ejemplo:

```
a=0
while 1:
    print(a)
    a+=1
```

Cómo que 1 es siempre 1, la condición se cumple indefinidamente. Podemos usar también while True: Estas construcciones son habituales con la instrucción break, cómo veremos algo más adelante cuando hablemos de los ficheros, en la sección 2.3.

2.2.d. Bucles for

Sirven para iterar sobre los elementos de una secuencia, por ejemplo, una lista:

```
a=["lunes", 'martes', 'miércoles', 'jueves', 'viernes', 'sábado', 'domingo']
for día in a:
    print(día)
```

Escribirá:

```
lunes
martes
miércoles
jueves
viernes
sábado
domingo
```

La función `range()` nos puede resultar muy útil para iterar sobre una secuencia de números. Por ejemplo:

```
for e in range(5):
```

```
    print 
```

`range(5)` devuelve una lista de 5 números, del 0 al 4, y de este modo la salida del programa es:

```
0
```

```
1
```

```
2
```

```
3
```

```
4
```

`range` nos permite indicar el elemento inicial, el final y el salto entre elementos:

```
range(start, stop[, step])
```

Entonces:

```
for e in range(2,27,3):
```

```
    print(e)
```

escribirá:

```
2
```

```
5
```

```
8
```

```
11
```

```
14
```

```
17
```

```
20
```

```
23
```

```
26
```

2.3. Trabajar con ficheros

2.3.a. Introducción

Muy a menudo la información que tratarán nuestros programas estará almacenada en archivos. Cuando procesamos textos, será necesario abrirlos, leer sus datos para procesarlos y cerrarlos. En esta sección veremos las instrucciones básicas necesarias para realizar las operaciones más habituales con archivos de texto

Cómo que los archivos de texto pueden estar en diferentes codificaciones de caracteres, ya desde el principio nos acostumbraremos a trabajar con el módulo `codecs`, que permite trabajar muy fácilmente con codificaciones de caracteres.

2.3.b. Abrir y leer un archivo de texto

Para abrir un archivo de texto en modo de lectura tendremos que cargar el módulo codecs y abrir el archivo de la siguiente manera:

```
import codecs

entrada=codecs.open("archivo1.txt","r",encoding="utf-8")
```

Fíjate que hemos especificado el modo de lectura ("r") y que el archivo está Unicode utf-8 ("utf-8"). Los archivos se pueden abrir en los siguientes modos:

- **r**: lectura
- **w**: escritura
- **a**: para añadir datos al final del archivo
- **r+**: tanto para leer cómo para escribir

En entrada ahora tenemos un objeto de tipo archivo que tiene tres métodos principales para leer el contenido del archivo:

- **read()**: lee todo el archivo
- **readline()**: lee una línea
- **readlines()**: lee todas las líneas

Vamos a practicar todo el relacionado con la lectura de archivos. Podéis descargar un archivo que se llama archivo.txt y que está codificado como Unicode utf-8:

Esta se la primera linia.

Esta se la segunda línea.

This is the third line.

Это четвертая строка.

これは、5行目です。

En el intérprete interactivo podemos probar las diversas opciones. Primero importamos codecs y abrimos el archivo en modo de lectura. Recordad que si no estás en el directorio donde se encontrar el archivo archivo.txt, tendrás que indicar la ruta completa al archivo.

```
import codecs

entrada=codecs.open("archivo.txt","r",encoding="utf-8")
```

La primera opción que tenemos es utilizar read() que leerá todo el archivo y lo pondrá en una cadena. Comprobémoslo:

```
>>> import codecs
>>> entrada=codecs.open("archivo.txt","r",encoding="utf-8")
>>> contenido=entrada.read()
>>> print(contenido)
Aquesta és la primera línia.
Esta se la segunda línea.
This is the third line.
```

Это четвертая строка.

これは、5行目です。

```
>>> type(contenido)
<type 'unicode'>
```

Nota: es posible que en una pantalla de Símbolo de sistema de Windows no se visualicen correctamente los caracteres cirílicos ni los japoneses. Es un problema de visualización, pero la lectura se lleva a cabo correctamente.

Otra opción es usar `readline()`, que te devuelve una línea:

```
>>> import codecs
>>> entrada=codecs.open("archivo.txt","r",encoding="utf-8")
>>> linea=entrada.readline()
>>> print(linea)
```

Aquesta és la primera línia.

```
>>> type(linea)
<type 'unicode'>
```

Si continuamos leyendo líneas:

```
>>> linea=entrada.readline()
>>> print(linea)
```

Esta es la segunda línea.

```
>>> linea=entrada.readline()
>>> print(linea)
```

This is the third line.

```
>>> linea=entrada.readline()
>>> print(linea)
```

Это четвертая строка.

```
>>> linea=entrada.readline()
>>> print(linea)
```

これは、5行目です。

Si ya se ha acabado el archivo y continuamos:

```
>>> linea=entrada.readline()
>>> print(linea)
```

Nos devuelve una cadena en blanco.

`readline()` a menudo se usa dentro de un bucle infinito con una condición que comprueba si se ha acabado el archivo. Esto lo encontrarás en el programa `programa-2-3-1.py`.

```
import codecs

entrada=codecs.open("archivo.txt","r",encoding="utf-8")

while 1:

    linea=entrada.readline()

    linea=linea.rstrip()

    if not linea:

        break

    print(linea)

entrada.close()
```

Esta solución se usa habitualmente cuando tenemos que tratar con ficheros muy grandes, puesto que leemos una línea cada vez y evitamos tener que cargar todo el archivo de texto en la memoria. Hemos introducido `rstrip()`, que elimina los espacios en blanco, incluidos los caracteres de salto de línea, del final de la cadena. También hemos introducido `close()` que sirve para cerrar el archivo una vez se acaba de leerlo.

`readlines()` lee todas las líneas del archivo y lo almacena en una lista. Podemos ver esta opción en el `programa-2-3-2.py`.

```
import codecs

entrada=codecs.open("archivo.txt","r",encoding="utf-8")

lineas=entrada.readlines()

print(lineas)

print(type(lineas))
```

`['Aquesta es la primera línia.\n', 'Esta es la segunda línea.\n', 'This is the third line.\n', 'Это четвертая строка.\n', 'これは、5行目です。\\n']`

```
type(linies)

<class 'list'>
```

Todavía tenemos alguna otra opción para leer archivos de texto:

Simplemente iterar sobre el propio objeto archivo (`programa-2-3-3.py`):

```
import codecs

entrada=codecs.open("archivo.txt","r",encoding="utf-8")

for linea in entrada:

    linea=linea.rstrip()

    print(linea)
```

Que nos da la salida:

Aquesta és la primera línia.

Esta es la segunda línea.

This is the third line.

Это четвертая строка.

これは、5行目です。

2.3.c. Escritura en archivos

Para escribir información en un archivo de texto simplemente hay que usar `write()`.

```
>>> salida=codecs.open("salida.txt","w",encoding="utf-8")
>>> salida.write("Escribimos una cadena de texto.\n")
```

Si abres el archivo `salida.txt` veréis que contiene:

Escribimos una cadena de texto.

Fíjate también que hemos añadido manualmente un salto de línea ("`\n`") al final de la línea.

2.4. Errores y excepciones

2.4.1. Introducción

Cuando programamos seguro que cometeremos errores e incluso programas correctamente programados pueden producir errores bajo ciertas circunstancias. Podemos distinguir dos tipos de errores principales: errores de sintaxis y excepciones.

Los errores de sintaxis se producen porque el código contiene algún tipo de error y el intérprete lo detecta y da un mensaje de error. A continuación observamos un ejemplo:

```
>>> print("Hola)
File "<stdin>", line 1
    print("Hola)
    ^
```

SyntaxError: EOL while scanning string literal

Fíjate que falta cerrar las comillas de `Hola` y por eso el intérprete avisa de este error. El mensaje de error que proporciona el intérprete es de gran importancia para corregir el error: indica la línea donde se produce el error y algún tipo de explicación del error. Hay que tener en cuenta, no obstante, que el error puede estar en alguna línea anterior a la que indica el intérprete.

A continuación mostramos un ejemplo de excepción (se trata del programa-2-4-1.py)

```
a=input("Introduce un número ")
b=input("Introduce otro número ")
c=float(a)/float(b)
print("La división es ",c)
```

Este programa funciona perfectamente en muchos casos, pero hay algunos en los que se produce error.

Por ejemplo, para 2 y b 3 funciona perfectamente:

Introduce un número 2

Introduce otro número 3

La división es 0.6666666666666666

Ahora bien, si por ejemplo al primer número no entramos un valor numérico, se produce una excepción:

Introduce un número w

Introduce otro número 3

Traceback (most recent call last):

File "programa-2-4-1.py", line 3, in <module>

```
c=float(a)/float(b)
```

ValueError: could not convert string to float: 'w'

O bien si el segundo número es un cero:

Introduce un número 4

Introduce otro número 0

Traceback (most recent call last):

File "programa-2-4-1.py", line 3, in <module>

```
c=float(a)/float(b)
```

ZeroDivisionError: float division by zero

Para evitar estos errores podemos escribir código adicional de forma que controlemos las posibles causas de error, como por ejemplo el programa-2-4-2.py.

```
a=input("Introduce un número ")
```

```
b=input("Introduce otro número ")
```

```
if a.isnumeric() and b.isnumeric() and not b=="0":
```

```
    c=float(a)/float(b)
```

```
    print("La división es ",c)
```

```
else:
```

```
    print("Las cifras que has introducido no son correctas")
```

Prueba este programa y verás que funciona bastante bien pero que solo admite números enteros, puesto que el método `isnumeric()` solo verifica si todos los caracteres que contiene la cadena son números. En la sección siguiente presentamos una manera mucho más efectiva de controlar los posibles errores que se pueden producir en un programa.

2.4.2. try...except

Con try...except podemos controlar las excepciones que se producen en un programa. Observemos el programa-2-4-3.py como podemos controlar los errores de este modo.

```
a=input("Introduce un número ")
b=input("Introduce otro número ")
try:
    c=float(a)/float(b)
    print("La división es ",c)
except:
    print("Las cifras que has introducido no son correctas")
```

Podemos hacer que nos muestre el mensaje de error que se produce importando el módulo sys y modificando la última línea (programa-2-4-4.py):

```
import sys
a=input("Introduce un número ")
b=input("Introduce otro número ")
try:
    c=float(a)/float(b)
    print("La división es ",c)
except:
    print("Las cifras que has introducido no son correctas",sys.exc_info()[0])
```